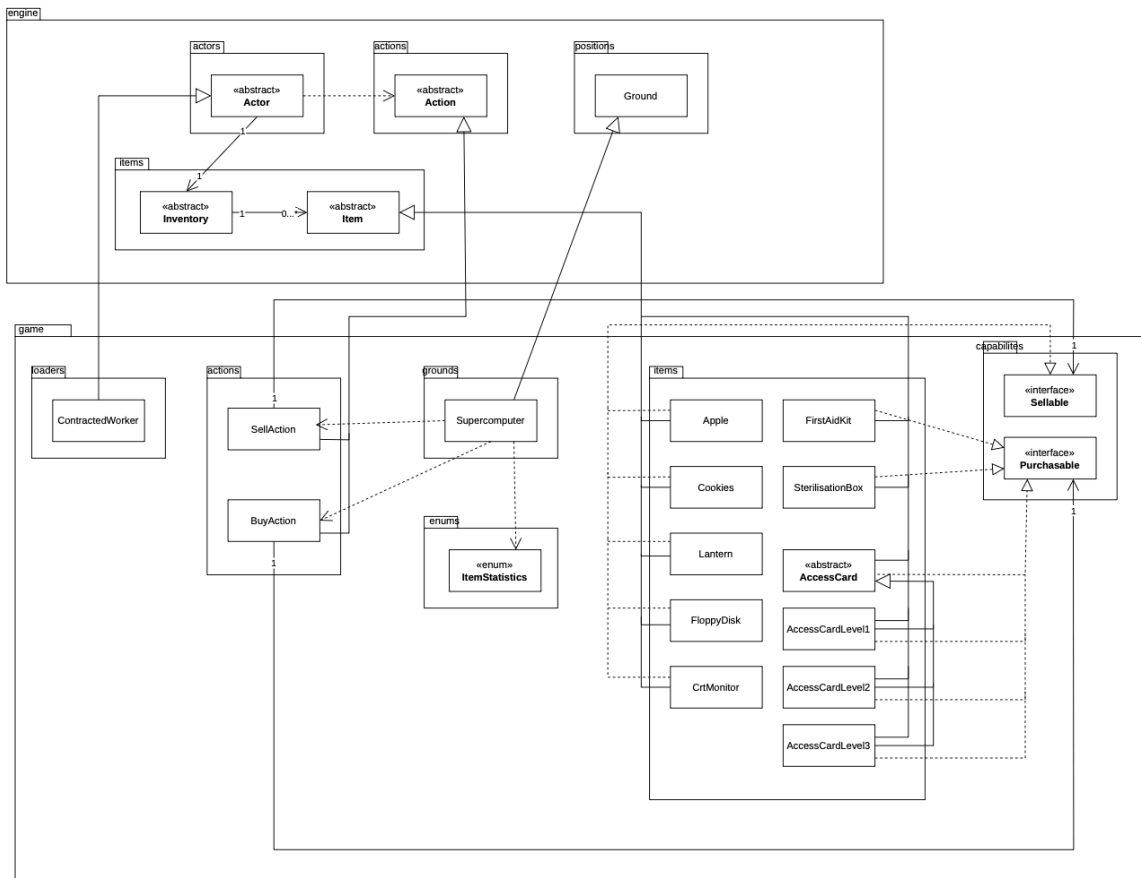


FIT2099 Assignment 2 Design Documentation

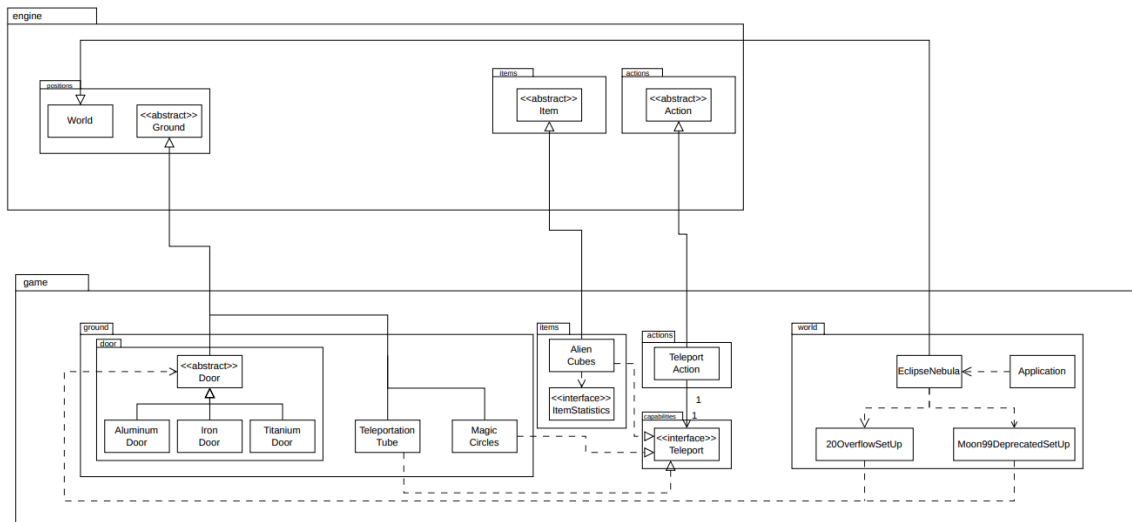
Design Diagrams

REQ1 UML Class Diagram:

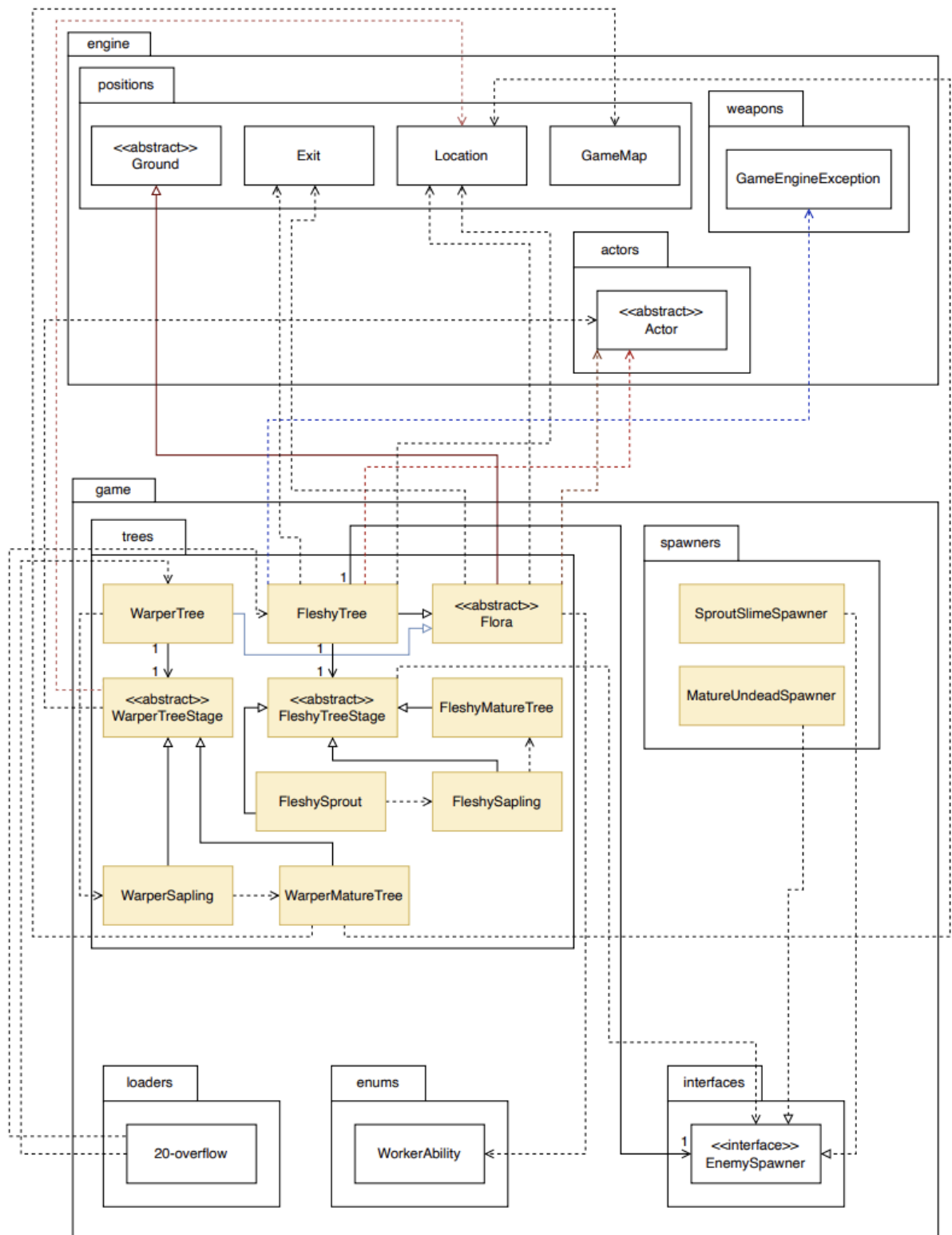


REQ2 UML Class Diagram:

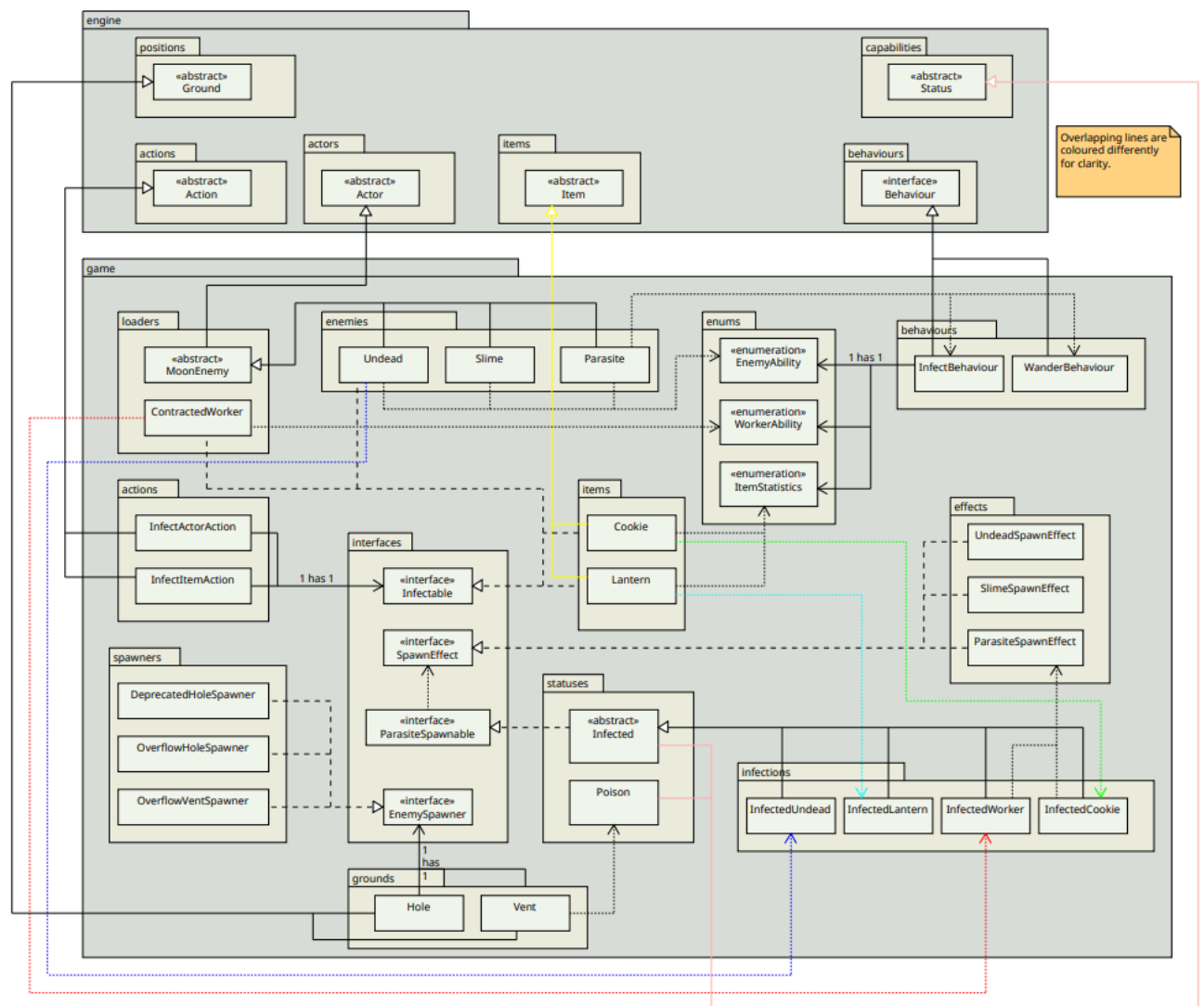
Note:
This UML diagram pertains to the
changes done for REQ 2.



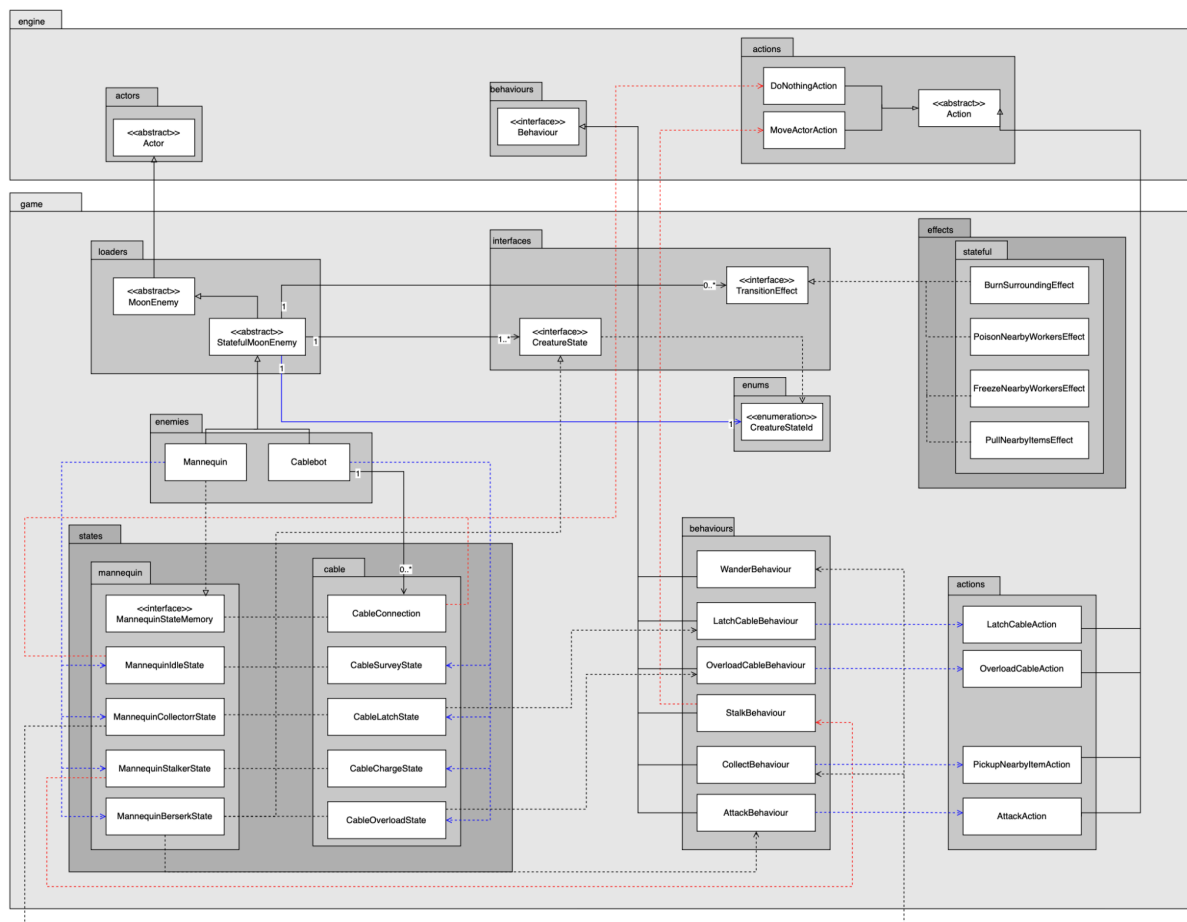
REQ3 UML Class Diagram:



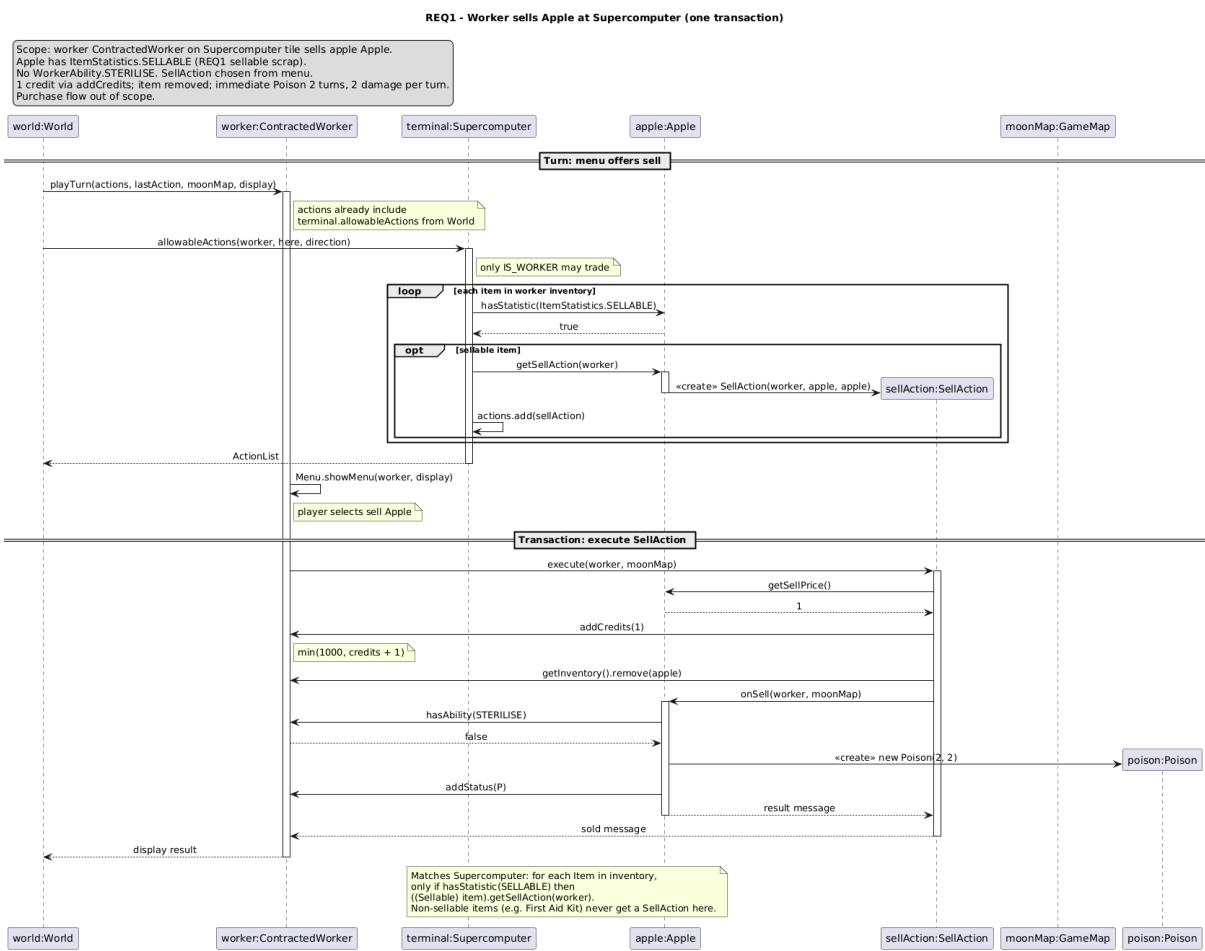
REQ4 UML Class Diagram:



REQ5 UML Class Diagram:



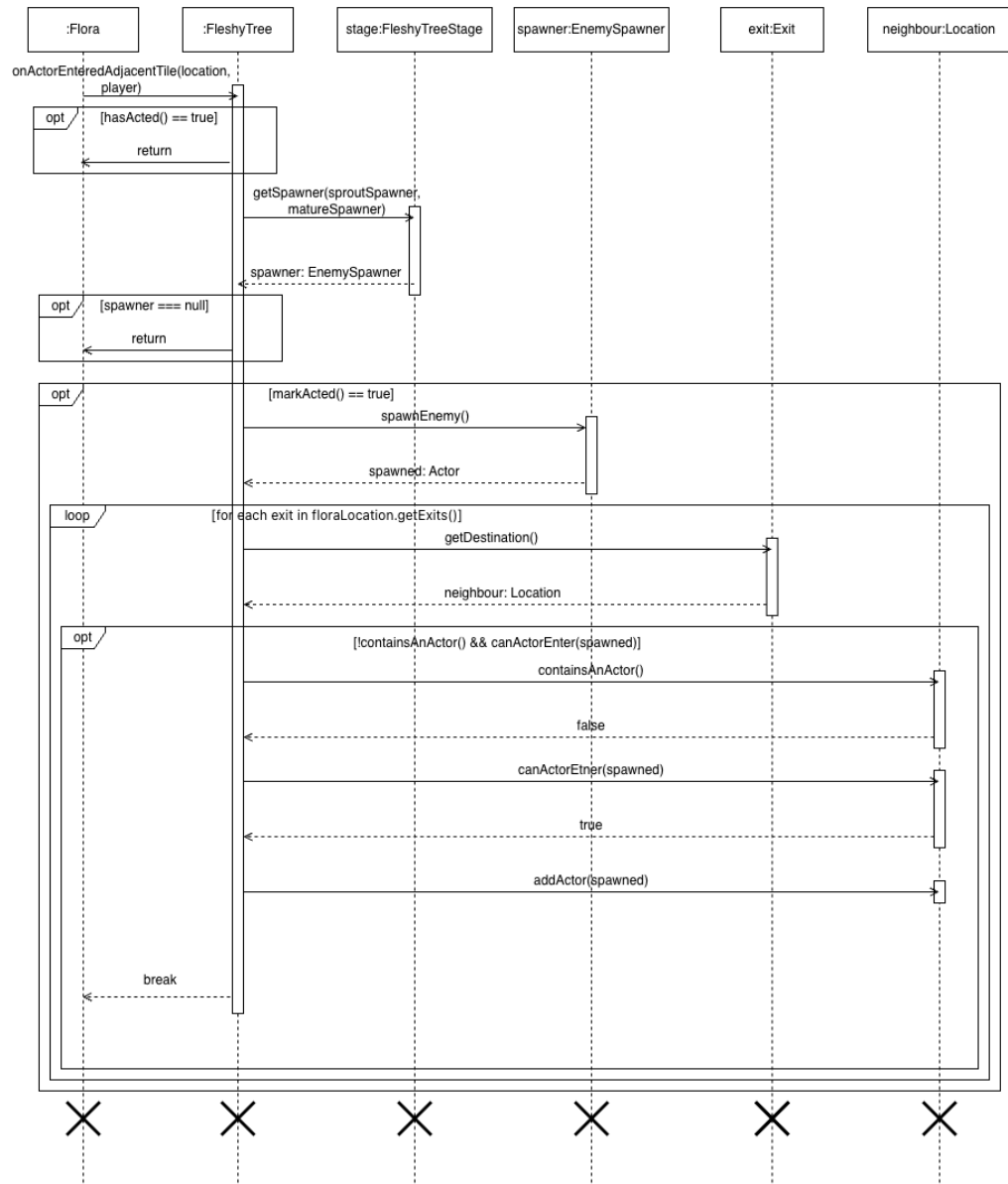
REQ1 UML Sequence Diagram:



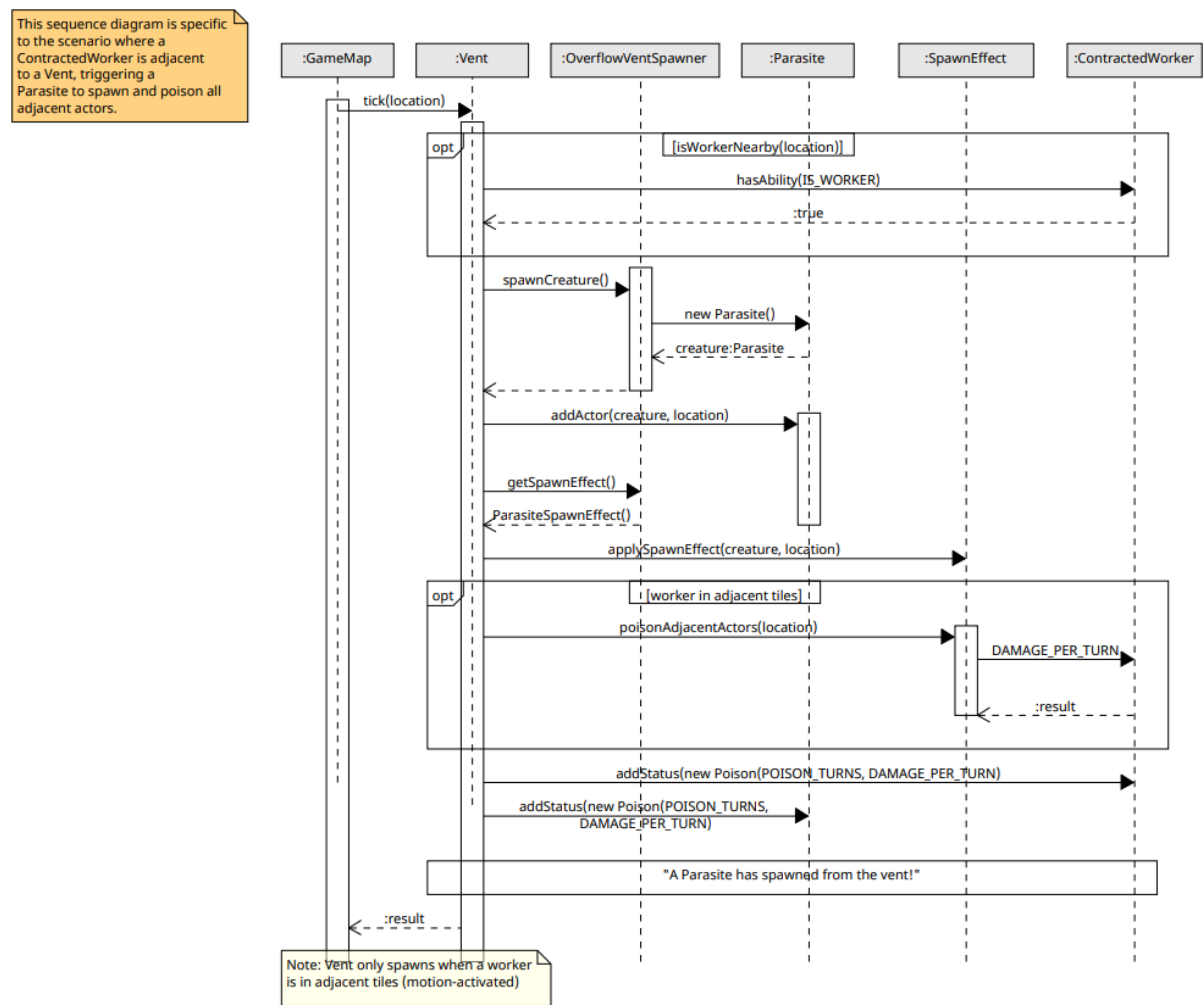
REQ2 UML Sequence Diagram:

REQ3 UML Sequence Diagram:

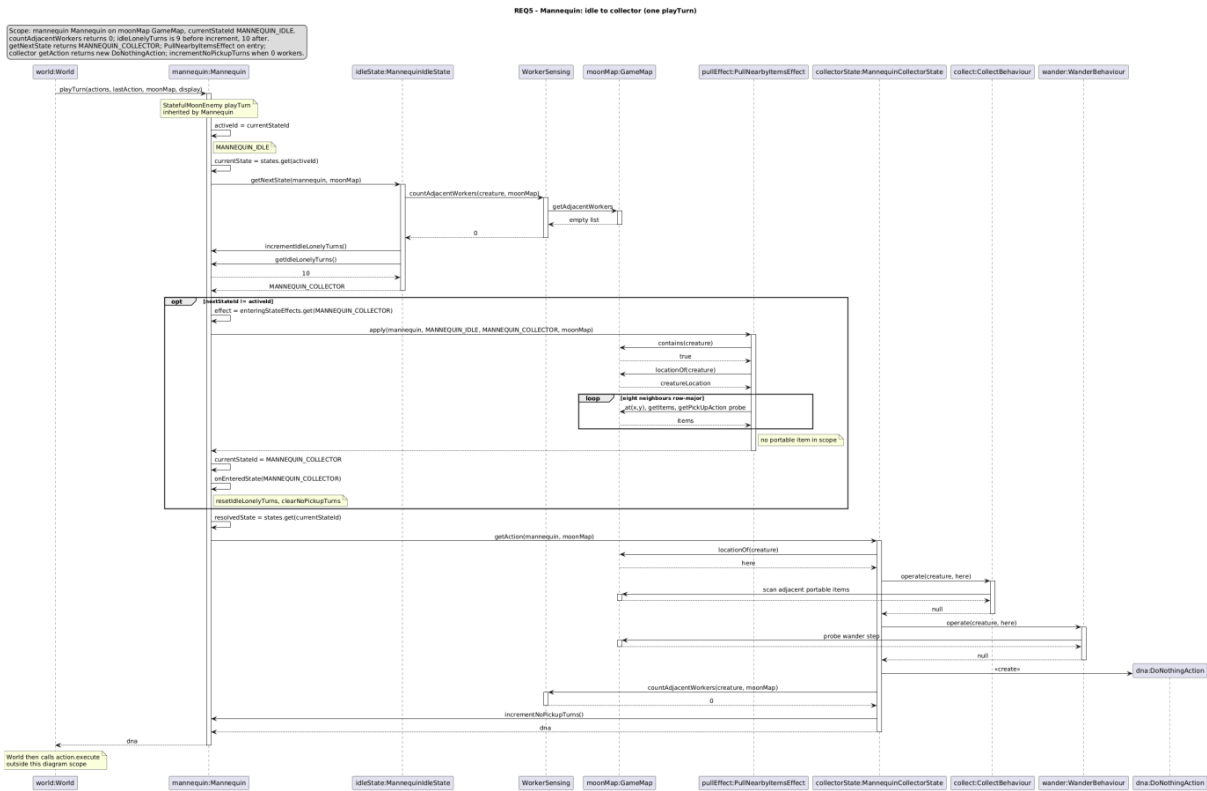
This sequence is specific to the scenario where the player steps on the surrounding of the FleshyTree



REQ4 UML Sequence Diagram:



REQ5 UML Sequence Diagram:



Design Rationale

REQ1: The Economy of Eclipse Nebula

a. Overall Trading System (Supercomputer, Buy/Sell Architecture)

Design 1: Direct logic inside items and Supercomputer (no abstraction)

Pros	Cons
Very simple initial implementation	Violates Open–Closed Principle (OCP), every new item requires modifying Supercomputer
No additional interfaces needed	High coupling between Supercomputer and all item types
Easy to understand for small scale system	Poor scalability as item types increase
No abstraction overhead	Repeated logic and large conditional blocks

This design tightly couples all item behaviour to a central class, making future expansion expensive and error-prone.

Design 2: Strategy-based item behaviour (Chosen)

Pros	Cons
Highly extensible (OCP compliant)	Slight increase in number of classes/interfaces
Each item encapsulates its own behaviour (SRP)	Requires consistent implementation across items
Supercomputer remains clean and generic	Some duplication in action creation logic
Easy to add new items without modifying engine	Requires careful interface design

This design best supports extensibility and keeps transaction logic modular and isolated per item.

b. Sellable / Purchasable Interface Design

Design 1: Single Transaction Interface

Pros	Cons
Less interfaces	Violates Interface Segregation Principle (ISP) because classes are forced to depend on methods they do not use
Simpler mental model	Cannot separate buy vs sell logic cleanly
Unified system	Harder to support asymmetric items

Buying and selling have fundamentally different effects in your game.

Design 2: Separate Sellable + Purchasable (Chosen)

Pros	Cons
Follows ISP (interfaces are role-specific)	More interface implementations per class
Supports asymmetric items (e.g., FirstAidKit only purchasable)	Requires dual interface handling in Supercomputer
Clear separation of responsibilities	

This aligns strongly with SOLID, especially ISP and SRP.

c. **BuyAction / SellAction Design**

Design 1: Logic inside Supercomputer

Pros	Cons
Fewer classes	Violates Single Responsibility Principle (SRP) because the Supercomputer is no longer just an interaction point
Centralized logic	Hard to maintain transaction rules
Easy to implement initially	Not reusable outside Supercomputer

logic becomes bloated and non-reusable.

Design 2: Action-based Transaction System (Chosen)

Pros	Cons
Strong SRP (each class handles one transaction type)	More classes introduced
Actions are reusable and decoupled	Slight indirection in execution flow
Easy debugging (each transaction isolated)	Requires passing dependencies (worker, item, interface)
Aligns with engine Action architecture	

Best fits existing action-based game engine design.

d. **Access Card Hierarchy Design**

Design 1: Enum-based clearance system only

Pros	Cons
Very simple	Not extensible
Minimal code	No item behaviour support
No side effects possible	Violates Open–Closed Principle because any new behaviour (such as damage on purchase, hidden fees, or special effects like inventory interaction) cannot be added without modifying the core logic

cannot support purchase effects like damage, fees, randomness.

Design 2: Abstract AccessCard class (Chosen)

Pros	Cons
Supports polymorphic purchase effects	More class files
Encapsulates clearance logic cleanly	Slight inheritance overhead
Extensible for new card types	Some duplication in constructors
Supports SRP per card type	

Required to support complex purchase side effects (damage, fees, randomness).

e. Door Access Control Design

Design 1: Hardcoded item checks in Door

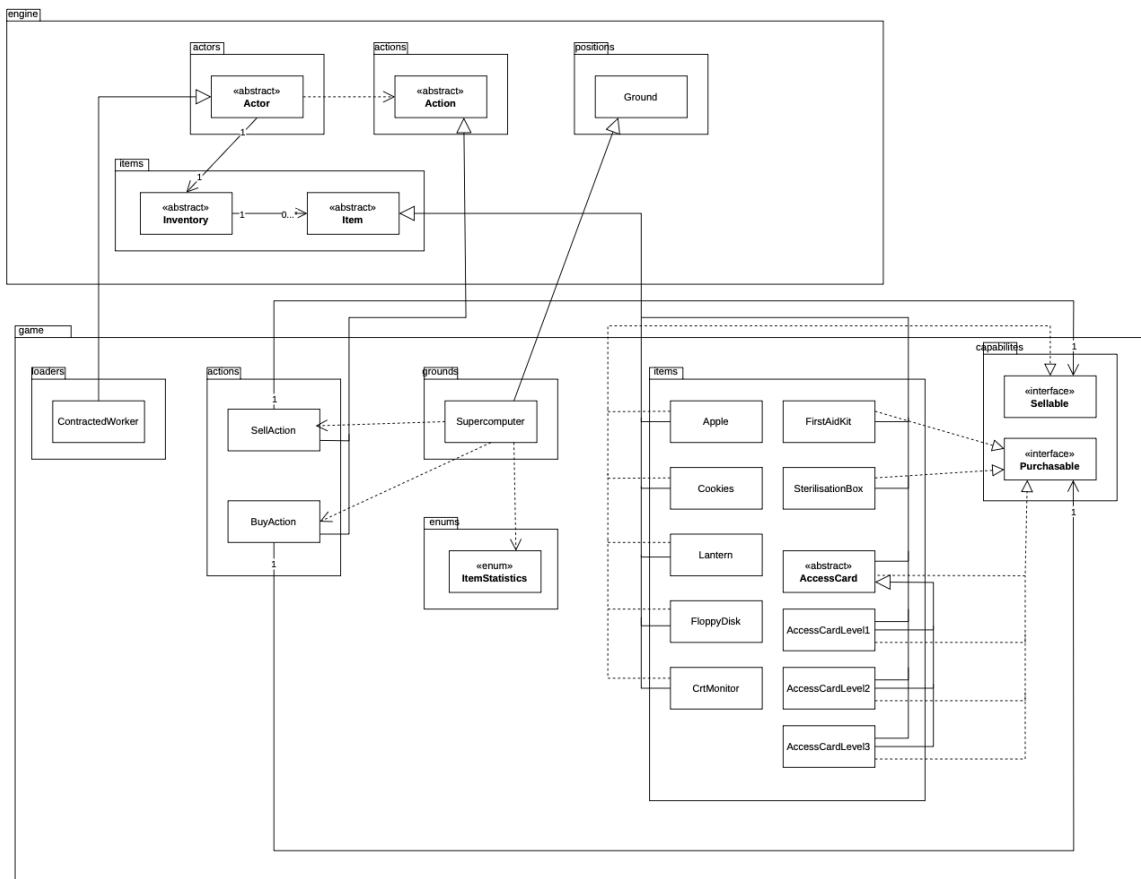
Pros	Cons
Simple implementation	Violates OCP because any new type of access card behaviour would require modifying existing conditional logic instead of simply extending the system through new classes
No abstraction needed	Cannot support new card types easily
Tight coupling	Poor scalability

cannot support purchase effects like damage, fees, randomness.

Design 2: Clearance-based system (Chosen)

Pros	Cons
Decouples Door from concrete card types	Requires consistent clearance system
Extensible (new cards don't modify Door)	Slight abstraction overhead
Supports OCP	
Works with polymorphism	

avoids Door needing to know specific card classes.



The final system is designed around a transaction-based architecture where buying and selling behaviours are cleanly separated into dedicated abstractions. Instead of embedding item-specific logic inside the Supercomputer, it delegates responsibility to Sellable and Purchasable interfaces, allowing each item to define its own pricing, side effects and transaction outcomes. The BuyAction and SellAction classes encapsulate the execution of all transactions, ensuring that all effects such as credit modification, health changes, status effects and environmental changes (like fire spawning) occur exactly at the point of interaction, rather than being scattered across multiple classes. The Supercomputer itself acts purely as an interaction gateway that exposes available actions to the player without directly managing item logic, reinforcing a clear separation of concerns. Access control through doors is handled using abstraction via clearance levels in access cards, while inheritance is used to specialise different card levels with unique effects and risks. This design follows SRP by separating responsibilities between items, actions and interaction points. OCP by allowing new items and behaviours to be added without modifying existing transaction logic. ISP by splitting selling and purchasing capabilities into distinct interfaces. And DIP by ensuring the Supercomputer depends on abstractions rather than concrete item implementations, resulting in an extensible and maintainable system.

REQ2: Teleportation

- a. Implementation of each Door type (Aluminium, Iron and Titanium doors)

Design 1: Create 3 subclasses of the doors without abstracting

Pros	Cons
Able to implement only to the intended ground class	A lot of repeated code, each class represents Ground and has the same attributes, only required level and character representation is different.
	Not utilizing the given engine, the abstract class of Ground in engine already gives methods and attributes for the doors

Design 2: Create an abstract class that represents doors and make 3 subclasses each door type.

Pros	Cons
Able to have methods with the same name be utilised by multiple subclasses.	
Less repeated code like for constructors or methods.	
Polymorphism is possible, where door could be written in code but any of the 3 door subclasses could be used.	

REQ3: Alien Flora

a. Implementation of each Flora FleshyTree and WarperTree

Design 1: Create concrete classes that represent the FleshyTree and WarperTree without extending any super classes.

Pros	Cons
Simpler & flexible, able to only implement the functionality that the respective actors need without the need of inheriting functionality.	Much repeated code, both classes represent Ground and have the same traits, only the character to represent them is different.
	Not fully utilizing the given engine, abstract class Ground in engine already gives the base methods and attributes for a Flora in the game.

Design 2: Create concrete classes that represent the FleshyTree and WarperTree by extending the given abstract class Ground in the engine.

Pros	Cons
Inheriting from abstract Ground class allows reusability of existing code and less repeated code and clearer implementation of FleshyTree and WarperTree class.	Inheriting a Ground abstract class introduces tight coupling between the FleshyTree and WarperTree concrete class and the Ground abstract class, changes to the abstract class may result in modification of all its subclasses, increasing the need of code maintenance.
Future extension of common methods only needs to be added to the abstract super class, child classes will inherit the methods with no need to modify child classes.	Some responsibilities are shared between two Flora and creates duplicated codes and violates Don't Repeat Yourself Principle.

Design 3: Create an abstract class Flora by extending the given abstract Ground class, which would be the base class of a Flora. Then, create concrete classes FleshyTree and WarperTree that extend abstract class Flora.

Pros	Cons
Single Responsibility Principle is managed well through the Flora abstract class for Tick method that handles Flora to act each turn, and their child classes handle their own identity.	If we want to add a new type of Flora that does not have to check the actors on the adjacent tile, we will have to modify the Flora abstract class for the onActorAdjacentTile abstract method.
Open and Closed Principle is well kept when we want to add a new type of Flora, we just create a new concrete	

class that inherits Flora abstract class, so no existing code is modified to extend the program.	
FleshyTree and WarperTree substitutes for Flora without causing an issue because tick, tryGrow and onActorAdjacentTile method aren't changed and not empty so Liskov Substitution Principle is well kept.	

b. Growing the next stage of each flora

Design 1: Create an Enum class for each flora that takes one character to represent the stage and use switch case statements to check their stage in their concrete classes.

Pros	Cons
Simple to implement as all logics are in one place in the Flora concrete class.	Violates Open and Closed Principle as it requires to modify the existing code when adding a new stage for the Flora.
	Violates Single Responsibility Principle as Flora concrete class takes too many responsibilities for each stage such as Spawning creatures and Warping actors.

Design 2: Create an abstract class FleshyTreeStage and WarperTreeStage that represents each stage in each Flora. Then, create concrete classes for each stage in each Flora. The abstract class that represents Flora's stage will contain abstract methods to make it consistent between stages for their responsibilities as well as the displayChar class attribute to represent the character to be displayed on the map.

Pros	Cons
Each concrete class that represents the stage of the Flora handles their own responsibility in each concrete class so it meets the Single Responsibility Principle.	Some stages such as the tryGrow method at the last stage of the Flora do not require to implement responsibilities but still need to implement them since they are abstract methods.
When we add a new stage of the Flora, we just create a new concrete class that inherits the Flora's stage abstract class to extend the stage of the Flora so no existing code is touched which meets the Open-Closed Principle.	
Having abstract class for each Flora's stage will	

Design 3: Create an Enum class for each flora that takes one character to represent the stage and handles the growing features in the Enum class as well as their responsibilities such as spawning creatures and teleporting adjacent workers.

Pros	Cons
The Open and Closed Principle is well kept when we want to add new stages to the Flora, we just add an Enum constant in the Enum class, so we don't have to modify the existing code to add a new stage to the Flora.	Enum class is supposed to contain the list of named constants and having methods in each constant is not a good design and unconventional.
Flora concrete class delegates its responsibilities to the Enum class, so it avoids using instanceof and switch case statements.	One Enum class handles all the stages of the Flora so it violates the Single Responsibility Principle.

c. Flora FleshyTree spawning creatures when a worker is at its surroundings

Design 1: FleshyTree concrete class handles detecting a worker in the surroundings of the FleshyTree and spawns the corresponding enemy from the passed spawner objects from instantiation and calling the method in the stage to receive the correct spawner.

Pros	Cons
FleshyTree takes EnemySpawner type of parameter so when we want to extend the feature to spawn another creature in the future, we don't have to modify the existing code but when we instantiate the object, we just pass in in the parameter itself so Dependency Inversion Principle is well kept here.	It might not be intuitive to the developers as to why the FleshyTree object requires two spawners to pass in to initialize before understanding the responsibility of the FleshyTree.
FleshyTree concrete class handles detecting the worker at its surrounding tiles and spawns the enemy according to their stage of the FleshyTree which meets the Single Responsibility Principle.	

Design 2: Each stage of the FleshyTree handles detecting a worker in the surroundings of the FleshyTree and spawning slime and undead at Sprout and Mature stage each.

Pros	Cons
Simpler to instantiate as no parameters are needed to pass in.	FleshyTree instantiates SlimeSpawner and UndeadSpawner inside the concrete class, so when we want to extend the feature to spawn another

	creature in the future, we have to modify the existing code, so the Dependency Inversion Principle is violated.
	Detecting a worker at its surroundings feature is going to be duplicated in each stage of the FleshyTree which violates DRY.

d. **Warping a worker into the random location of the map**

Design 1: Implement the warp method inside the WarperTree concrete class

Pros	Cons
Simple to implement and easier to find the warp method as it stays in the concrete class itself.	Single Responsibility Principle is violated because WarperTreeStage should handle the warp feature since its different stage has their own responsibilities when the actors are on the adjacent tiles.

Design 2: Implement the warp feature by overriding the abstract class WarperTreeStage's abstract method onActorAdjacent in the WarperMatureTree concrete class.

Pros	Cons
Single Responsibility Principle is well kept here because warp feature is only doable in the WarperMatureTree stage since its different stages have their own responsibilities when the actors are on the adjacent tiles.	Some stages do not require to do anything when the worker is on adjacent tiles but still have to implement them since they are abstract methods.
Utilizes the GameMap class to get the random location of the map so when there is another map added in the future such as 20-overflow will still make it work and not depend on one map itself.	

e. **Flora reacting only once per turn**

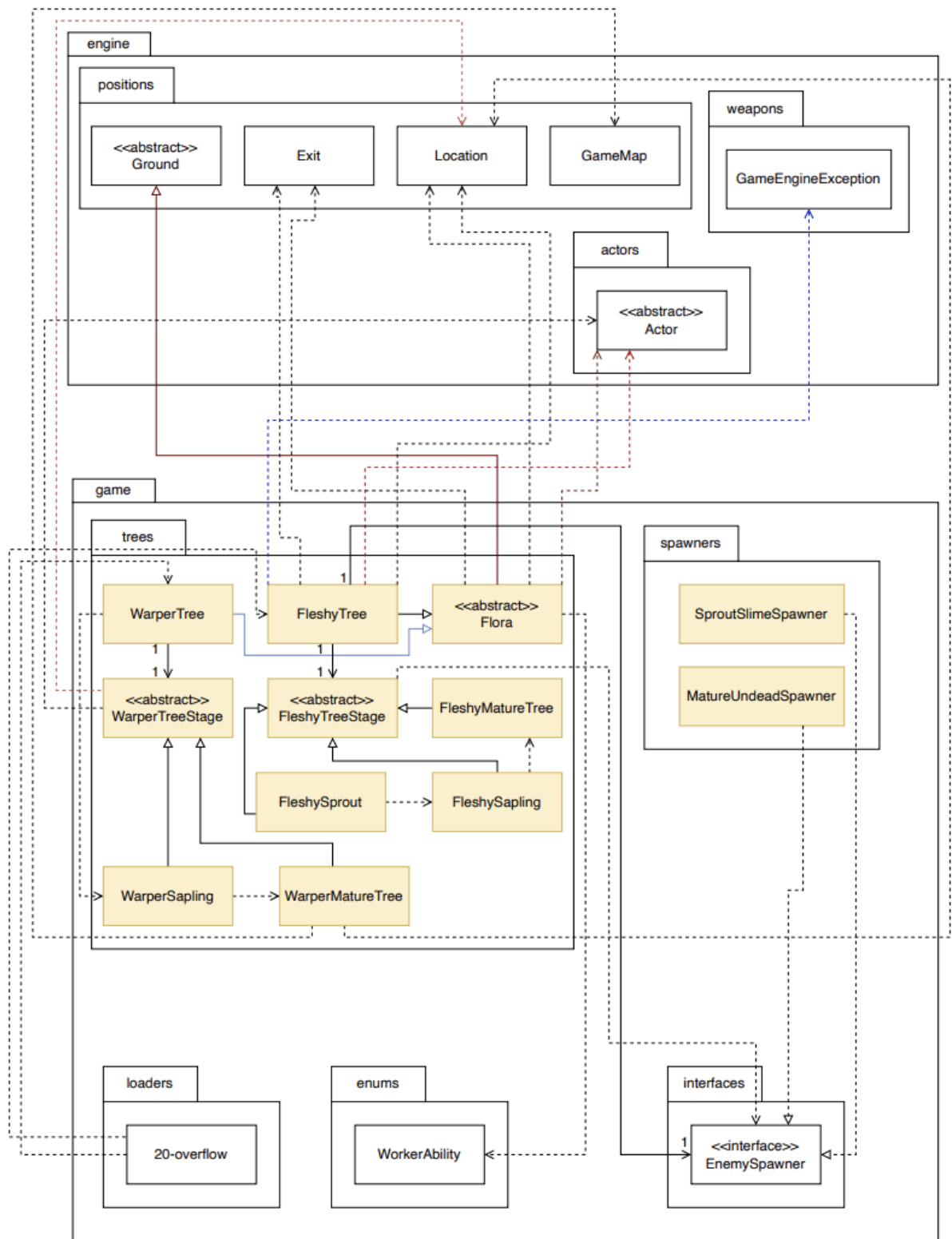
Design 1: Implement reacting only once per turn feature in the concrete class itself

Pros	Cons
Simpler & flexible, able to only implement the functionality that the respective grounds need without the need of inheriting functionality.	It violates Don't Repeat Yourself Principle as it must implement the feature that guards the double action for growth and its responsibility acting at

	the same turn and if the implementation between Flora is different, it will bring an inconsistency.
	It brings more complexity in implementing the feature as it already exists as a feature that can be used from the game engine.

Design 2: Utilize its parent's class Tick method to react only once per turn with ActedThisTurn attribute that restricts growing and acting at the same time

Pros	Cons
Single Responsibility Principle is well kept here because Flora abstract class manages the double actions on one turn using ActedThisTurn as a safeguard and subclasses do not have to handle this anymore.	The ActedThisTurn attribute is shared in the Flora base class and requires subclasses to understand the markActed and hasActed methods to use correctly if the subclasses are going to use them in the future.



The diagram above shows the final design of requirement 3.

Both FleshyTree and WarperTree created the abstract Flora class which extends the game engine's abstract Ground class. Since these concrete classes share many common attributes and methods such as age tracking and tick lifecycle, it is logical to abstract these identities to avoid repetitions in the code (Don't Repeat Yourself). The

new concrete classes do not modify the methods in the abstract Flora class which makes them have the same attributes and methods and enables them to substitute its base class (Liskov Substitution Principle). Stage logic is abstracted into the FleshyTreeStage and WarperTreeStage abstract classes. They handle the flora's growth, creature spawning and actor warping features. Each abstract class manages to grow the stage based on the age and the random chance and to perform their own responsibilities in their stage. Having them in the abstract class allows no need to modify the existing code when adding new stages since we can just create a new stage concrete class that inherits their flora's stage abstract class (Open-Closed Principle). Each concrete class that inherits their flora's stage abstract class is responsible for their own stage (Single Responsibility Principle). FleshyTree takes two EnemySpawner type parameters in the Constructor to instantiate the object rather than instantiating the spawner in their own concrete class so this makes FleshyTree class has no dependency with spawner as they can just pass in different type of spawner without modifying its existing concrete class code (Dependency Inversion Principle).

REQ4: New Creature Detected

A. Implementation of Parasite and Infection System

Design 1: Use instanceof and downcast in InfectBehaviour to identify the infectable targets and apply infection effects

Pros	Cons
Simple and straightforward to implement and easier to understand.	Adding new infectable targets requires modifying InfectBehaviour with additional instanceof checks which violates Open-Closed Principle (OCP)
No additional classes or interfaces needed.	Overuse of instanceof and downcasting is a code smell
	High coupling as InfectBehaviour would depend on ContractedWorker, Undead, Lantern and Cookie

Design 2: Create an Infectable interface with getInfectedStatus() and getInfectAction() methods, using asCapability() from the engine to identify infectable targets without instanceof or downcasting.

Pros	Cons
No instanceof or downcasting needed, as the game engine asCapability() handles the type checking.	All infectable targets to implement the Infectable interface correctly.
Helps with having Open-Closed Principle as new infectable targets just implements Infectable and InfectBehaviour won't change.	User must understand how asCapability() works and how Optional is used.
Lower coupling as it does not depend on specific concrete classes.	
Polymorphism will handle different infection effects – ContractedWorker will return InfectedWorker, Undead returns InfectedUndead, etc.	

B. Implementation of Infected Status Effects

Design 1: Handle all infection effects directly inside InfectBehaviour or InfectActorAction/InfectItemAction

Pros	Cons
All infection logic in one place.	Disregards Single Responsibility Principle as InfectBehaviour will manage both finding targets and applying the effects.

Design 2: Create an abstract Infected class implementing Status, with subclasses for each infection effect (InfectedWorker, InfectedUndead, InfectedLantern and InfectedCookies).

Pros	Cons
Each subclass manages its own infection effect independently.	Developer must understand how the engine calls tickStatus()
New infection effects just extend Infected, existing classes never change.	

C. Implementation of ParasiteSpawnable Interface

Design 1: Duplicate spawnParasite() logic in every class that needs to spawn Parasites.

Pros	Cons
No extra interface needed.	Harder to maintain as changing spawn logic requires modifying every class that duplicates it.

Design 2: Create a ParasiteSpawnable interface with a default spawnParasite() method that all classes needing to spawn Parasites.

Pros	Cons
Defines once in the interface and can be reuse by all the implementing classes e.g. (InfectedWorker, InfectedCookie).	Harder to maintain as changing spawn logic requires modifying every class that duplicates it.
Lower coupling as spawnParasite() is self-contain and does not depend on other specific class.	

D. Implementation of Spawning System for Hole and Vent

Design: Create an EnemySpawner interface with spawnEnemy() and getSpawnEffect() methods, with separate spawner classes per map for DeprecatedHoleSpawner, OverflowHoleSpawner and OverflowVentSpawner.

Pros	Cons
When adding new maps, we can just add the new spawner class, Hole and Vent never changed. (OCP)	More classes would be needed
Easily extensible for future implementations or developments. New spawners just implement EnemySpawner.	Ensure spawnEffect() returns correctly.

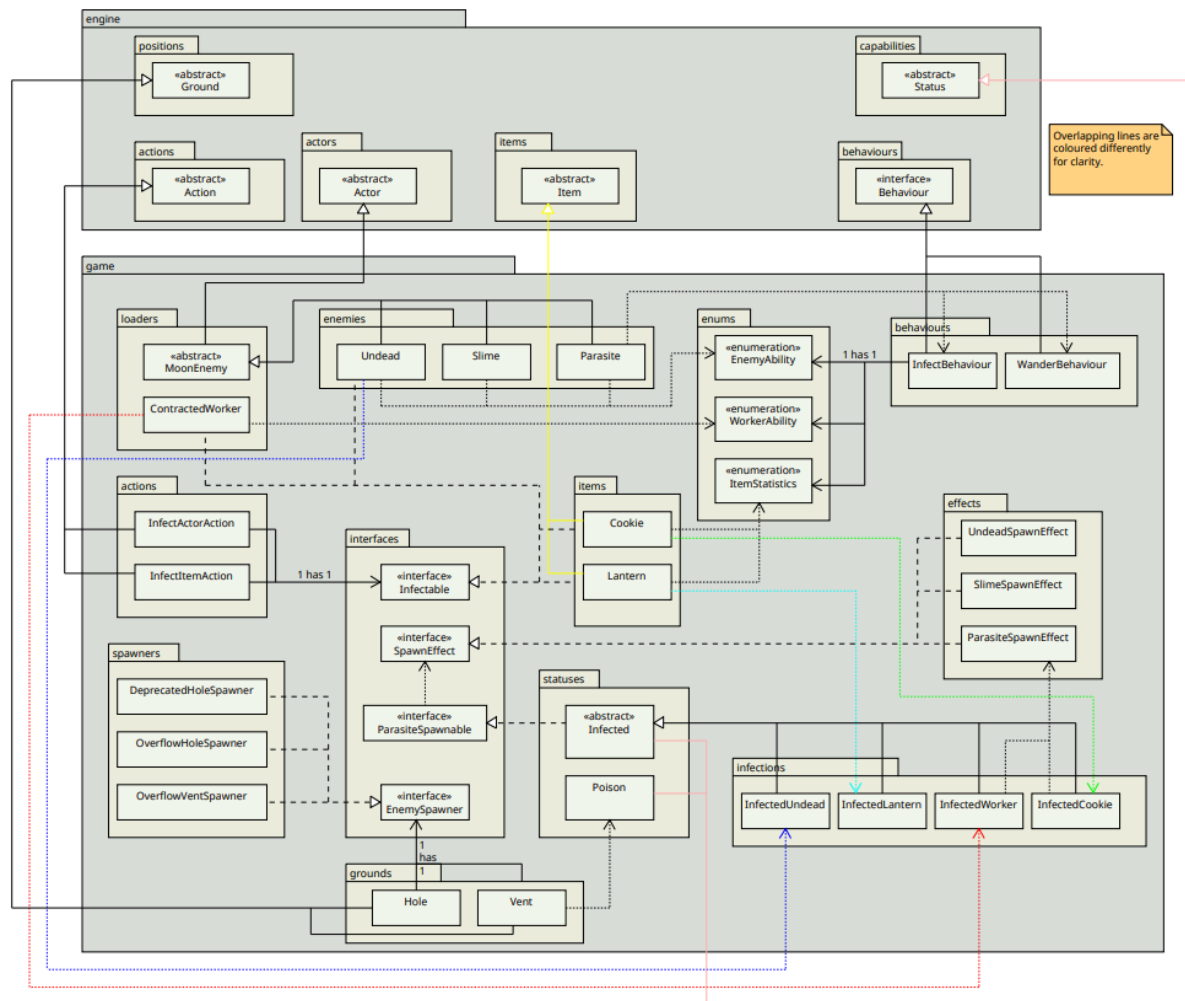
E. Implementation of Spawn Effect

Design 1: Handle spawn effects directly inside Hole and Vent using instanceof checks to identify the spawned creature type.

Pros	Cons
No extra classes needed.	Hole and Vent would now have to occupy more methods for spawning effects.
	Harder to maintain.

Design 2: Create a SpawnEffect interface with applySpawnEffect() method for UndeadSpawnEffect, SlimeSpawnEffect and ParasiteSpawnEffect each implement their own effect.

Pros	Cons
Spawn effect logic defined once per effect class, reused by Hole, Vent and infection spawning.	Requires spawners to be correctly stored and return from getSpawnEffects() properly.
Easily extensible for future implementations or developments. New spawn effects can just implement SpawnEffect without touching Hole or Vent.	



The diagram above shows the final design of requirement 4, which utilizes designs of part A, B, C, E with Design 2 and part D stated above to implement Parasite and Infection System for the new enemy. Infected Status Effects for infected actors/items. Parasite Spawnable interface for spawnParasite() logic. Spawning system for Hole and Vent. Lastly, Spawn Effects for enemies.

The new Parasite class extends the abstract MoonEnemy class, Infected class implements the Status and ParasiteSpawnable, InfectActorAction and InfectItemAction classes extend to the abstract Action class. Infected extends to InfectedWorker, InfectedUndead, InfectedCookie, and InfectedLantern. All these classes inherit their methods and attributes from these existing abstract classes. We do this to avoid repetition in code (DRY).

The infectable interface provides a clear contract of which actors and items can be infected by a Parasite, with getInfectedStatus() and getInfectAction() methods that each infectable class implements to return its own correct infected status and action. Using asCapability() from game engine's GameEntity, gives us better and easier functionality without modifying any existing code (Open-Closed Principle).

The `ParasiteSpawnable` interface provides us a default `spawnParasite()` method that is reused by both `InfectedWorker` and `InfectedCookie`, this avoids duplicate Parasite spawning logic across multiple classes (DRY). Both of these classes implements the `ParasiteSpawnable` which promotes code reusability and maintainability.

The `DeprecatedHoleSpawner`, `OverflowHoleSpawner`, and `OverflowVentSpawne` implements the `EnemySpawner` interface, this removes dependencies between two concrete classes without changing the flow of the logic and by diverting the dependency of having two concrete classes, it depends on `EnemySpawner` (Dependency-Inversion Principle). This also increases extensibility for code usage and creates better maintainability for future developments.

The `ParasiteSpawnEffect`, `SlimeSpawnEffect`, `UndeadSpawnEffect` implements the `SpawnEffect` interface. This ensures that the spawn effects, whenever a creature enter the map from a spawnpoint, it can be apply consistently throughout all spawning scenarios like Hole and Vent. The infection spawning also applies and calls the `applySpawnEffect()` the same way. This promotes better code reusability and avoids duplicate spawn effect logic across multiple classes (DRY and Open-Closed Principle). Lastly, using `EnemyAbility`, `WorkerAbility`, and `ItemStatistics` where they all have `IS_INFECTABLE` instead of `instanceof` in all spawn effects classes so that we can identify workers, enemies and items. This avoids downcasting and reduces dependencies on concrete classes, which gives us easier and better extensibility without modifying any existing spawn effect logic/code (Open-Closed Principle).

REQ 5: Stateful Creatures

a. Implementing the states for each creature

Design 1: Store the current state as an enum inside Mannequin / Cablebot and use large switch statements in playTurn() for both transitions and ordinary actions.

Pros	Cons
Fewer classes as all logic is visible in one file and is easy to trace for a single creature.	playTurn() grows quickly and mixes transition rules, entry effects, and per-state actions.
	Adding more states or creatures in the future forces edits to the same method, reducing extensibility (violates Open-Closed).

Design 2: Introduce CreatureState interface (one concrete class per mode, e.g. MannequinIdleState, CablebotSurveyState) (Final design)

Pros	Cons
Each state class owns only its transition rules and ordinary action (Single Responsibility Principle).	More classes and wiring in constructors
New states or creatures extend the system without rewriting the template in StatefulMoonEnemy's playTurn.	
Transition graph is explicit per class as deterministic successors are easy to mark and test.	

b. Immediate effects on state transition

Design 1: Put entry logic inside each CreatureState's getNextState or getAction.

Pros	Cons
All behaviour for a mode is in one class	Entry effects would run even when the creature stays in the same mode if coded carelessly, or duplicate checks across states.
	Reusing the same effect (e.g. pull items) across multiple states duplicates code.

Design 2: Introduce TransitionEffect interface and register effects with StatefulMoonEnemy's addEnteringStateEffect (Final design)

Pros	Cons
Clear separation: transition consequence vs ordinary behaviour.	
Same effect class can be reused without duplication (PullNearbyItemsEffect on Mannequin collector/stalker and Cablebot survey).	

c. Storing transition context (counters, cables)

Design 1: Put all fields (idleLonelyTurns, noPickupTurns, charge, cableConnections) only in the enemy classes)

Pros	Cons
Single object holds all mutable data.	States depend on concrete enemy types making it harder to test states in isolation.
	Easy to break encapsulation with public fields everywhere.

Design 2: Keep creature-specific data on the enemy (MannequinStateMemory for idle/no-pickup streaks and Cablebot keeps CableConnection) (Final Design)

Pros	Cons
States depend on abstractions (StatefulMoonEnemy and small memory interface), not every field.	Some downcasting to Cablebot in its states (minimal for creature-specific graph).
onEnteredState on Mannequin resets streaks in one place when any transition commits.	

d. Counting adjacent workers

Design 1: Duplicate neighbour-scan loops in every state and effect.

Pros	Cons
All logic is visible in one class.	Repeated logic and inconsistent scan order for tie-breaking, reduces maintainability and violates DRY.

	Changing adjacency rules would require editing many classes, violating OCP.
--	---

Design 2: Create a separate helper class WorkerSensing (Final Design)

Pros	Cons
One class counts adjacent workers. (SRP).	It is a static utility and not injected.
Mannequin/Cablebot/effects depend on sensing API, not each other's loops (OCP).	

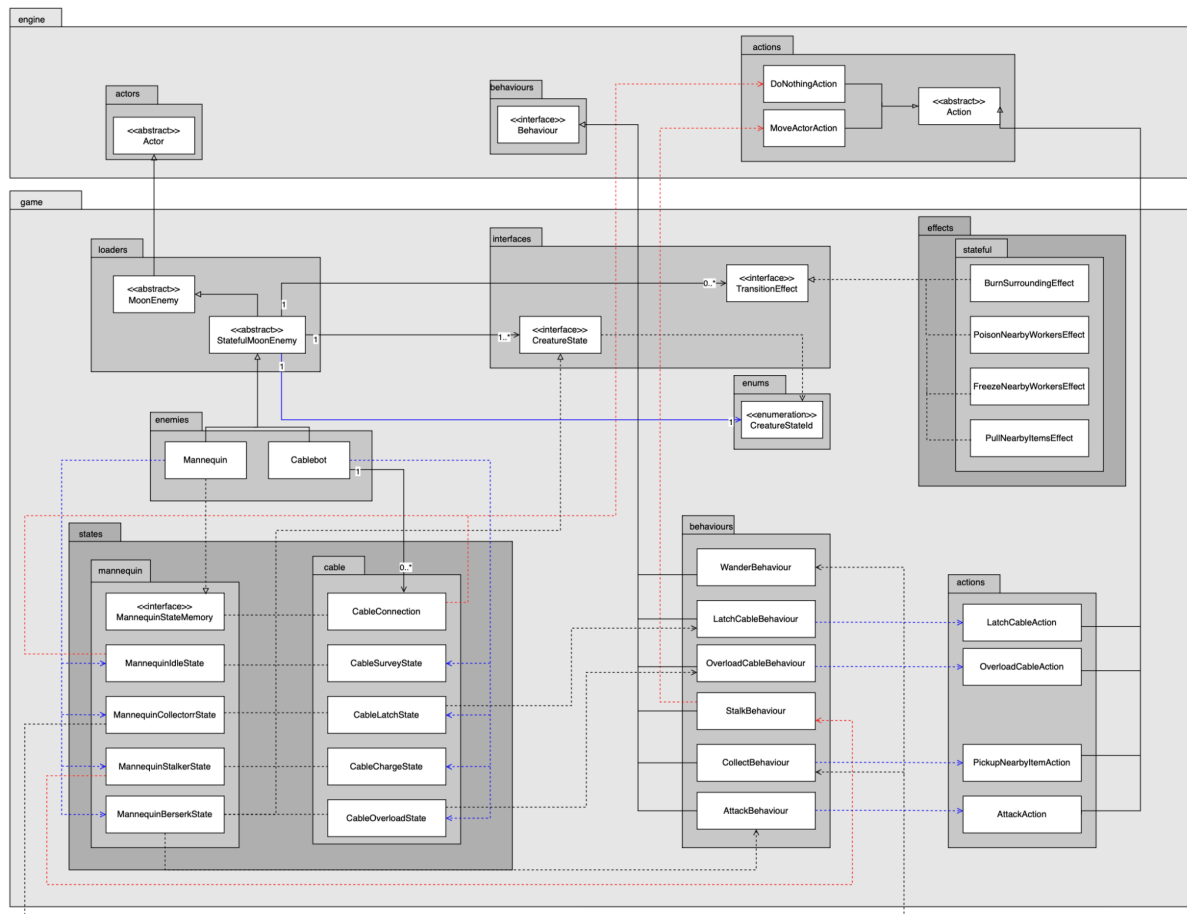
e. New Behaviours

Design 1: Implement new behaviours pickup, stalk, latch, overload entirely inside each state's getAction.

Pros	Cons
No extra behaviour classes.	Classes become large, violating the Single Responsibility Principle.
	Reusing "pick up from ground" or "step toward target" requires copy-paste.

Design 2: Implement new behaviours that extend Behaviour (CollectBehaviour, StalkBehaviour, LatchCableBehaviour, OverloadCableBehaviour).

Pros	Cons
Each state class only decides when to change mode and which behaviour(s) to use that turn. Each behaviour class owns one piece of complex logic (SRP).	
New stalk/latch rules extend behaviour classes without touching unrelated states (OCP).	
Behaviours plug into existing engine Behaviour<Actor, Action> and game system (LSP).	



The design applies the State pattern by representing each creature state as a separate class implementing the `CreatureState` interface. This allows each state to define its own transition rules and turn behaviour without placing all logic inside the creature class. The `MoonEnemy` abstract class only stores the current state and delegates state-specific decisions to the state object. Thus, `StatefulMoonEnemy` extends `MoonEnemy` to inherit enemy characteristics and have its own stateful characteristics.

The design follows the Single Responsibility Principle because state classes handle state decisions, behaviour classes handle turn actions, and transition effect classes handle the immediate consequences of changing state. For example, `MannequinIdleState` decides when the Mannequin should become idle, while `FreezeNearbyWorkers` handles the immediate effect that occurs when the Mannequin enters that state.

New complex behaviours required are delegated to Behaviour implementations rather than inlined in states. `MannequinCollectorState` composes `CollectBehaviour` and `WanderBehaviour`; `MannequinStalkerState` uses `StalkBehaviour`; `CablebotLatchState` and `CablebotOverloadState` use `LatchCableBehaviour` and `OverloadCableBehaviour`. States decide when to change mode and which behaviour to run while behaviour classes own the detailed map logic (ground pickup, greedy stalk step, cable attach/scoring, overload via cables). That split follows Open-Closed Principle: new rules can be extended in behaviour classes without rewriting the `StatefulMoonEnemy` structure.

The design follows the Open/Closed Principle because new states and transition effects can be added without modifying the core MoonEnemy and StatefulMoonEnemy class. For example, adding a new HidingState later would only require a new class implementing CreatureState.

The design follows Dependency Inversion because StatefulMoonEnemy depends on abstractions such as CreatureState and TransitionEffect, rather than depending directly on concrete state classes. Mannequin and Cablebot constructors act as the composition root: they addState and addEnteringStateEffect with concrete instances. Liskov Substitution Principle holds because any registered CreatureState can replace another in the map for getNextState and getAction without changing playTurn.

The design also avoids random state transitions. Every state transition is based on deterministic conditions such as the number of nearby workers. If multiple conditions are true, priority order is used to choose the next state consistently.

REQ5 introduces necessary connascence of execution order (transition effect before ordinary action) and connascence of name via StatefulCreatureStateId. Connascence of type appears where states cast to MannequinStateMemory or Cablebot, which is acceptable for two known creatures but would increase if many new stateful enemies were added. Connascence of values exists around transition thresholds (e.g. 10 lonely idle turns, 3 no-pickup turns); these are documented in the README and implemented as constants in state classes. Shared WorkerSensing and TransitionEffect registration reduce connascence of meaning and algorithm that would otherwise exist if every state duplicated neighbour-scan and entry effect code.